

DMS

Beginner Guide

DMS Project

July 2026

Learn C through the Distribution Management System

Welcome

This guide teaches you the **Distribution Management System (DMS)** project from zero. You only need basic C: **printf**, **scanf**, **for** and **while** loops, simple **arrays**, and **if/else**. Everything else is explained here as we go.

Think of DMS as a practice warehouse game in your terminal. Company holds big stock, dealers buy from company, shops (retailers) buy from dealers. You log in, view tables, and place orders.

What you will learn

- What the project does in real life (supply chain).
- How to compile and run the program on your computer.
- What every file does and how they connect.
- New ideas: functions, multiple files, strings, 2D arrays, extern.
- How ordering works step by step, with real numbers.

1. The Big Picture

1.1 Real-world story

A rice company keeps 100 bags in a central warehouse. A **dealer** in Dhaka orders 20 bags. A **shop** in Mirpur orders 5 bags from that dealer. Customers buy from the shop. DMS simulates this chain on your screen.

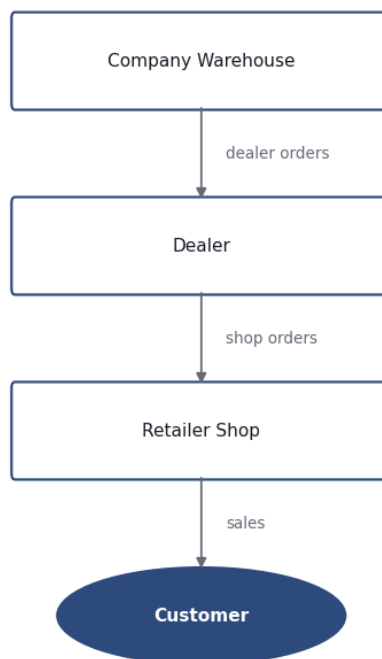


Figure 1: Products flow down the chain; orders go up

1.2 Three user roles

Role	Login	Main actions
Company	Password only	View all dealers, products, warehouse stock
Dealer	Dealer ID + password	View own stock, order from company
Retailer	Retailer ID + password	View own stock, order from dealer

Who can do what

Password for Company, Dealer, and Retailer login: 1122

2. Build and Run (Start Here)

2.1 Open terminal in project folder

BASH

```
cd /path/to/DMS
```

2.2 Compile all C files together

C projects often split code into many `.c` files. GCC compiles each file and links them into one program called **dms**.

BASH

```
gcc main.c data.c display.c company.c dealer.c retailer.c -o dms
```

Part	Meaning
gcc	The C compiler
main.c ... retailer.c	Six source files — all required
-o dms	Output executable named dms

Compile command explained

2.3 Run the program

BASH

```
./dms
```

2.4 Quick test session

1. Main menu: choose 2 (Dealer Login).
2. Enter Dealer ID: 1 (Dhaka Dealer).
3. Password: 1122.
4. Choose 3 (Order From Company), Product ID 1, Quantity 10.
5. Choose 0 to logout, then 0 again to exit.

If gcc is missing on Mac: run `xcode-select --install`

3. Project Files

Like chapters in a book — each file has one job. Together they form one program.

File	Job
main.c	Starts program, shows main menu
data.c / data.h	All products, dealers, retailers, stock data
display.c / display.h	Menus, tables, password, success/error messages
company.c	Company panel after login
dealer.c	Dealer panel and ordering from company
retailer.c	Retailer panel and ordering from dealer

File map

3.1 How files talk to each other

A **header file (.h)** lists what exists (function names, variables). A **source file (.c)** contains the real code. **#include "data.h"** at the top of a file means: use the shared data declared in data.h.

c

```
#include "data.h"    /* I need product lists and stock arrays */
#include "display.h" /* I need print_menu_item, print_error, etc. */
```

4. C Basics You Already Know

Before new ideas, here is how familiar syntax appears in DMS.

4.1 printf — show text

C

```
printf("Enter your choice: "); /* no newline at end */  
printf("Welcome, %s\n", dealer_name[dealer_index]); /* %s = string */
```

4.2 scanf — read input

C

```
scanf("%d", &choice); /* read integer into choice */  
scanf("%s", password); /* read word into char array */
```

4.3 for loop — walk through a list

C

```
for (i = 0; i < dealer_total; i++) {  
    print_table_dealer_row(dealer_id[i], dealer_name[i]);  
}
```

4.4 do-while — menu until user exits

C

```
do {  
    /* show menu */  
    scanf("%d", &choice);  
    if (choice == 1) list_dealers();  
} while (choice != 0); /* 0 = back or exit */
```


5. New Ideas in This Project

5.1 Functions

A **function** is a named block of code you can call. **void** means it returns nothing. **int** means it returns a number (like 1 for true, 0 for false).

C

```
void list_dealers() { /* does work, returns nothing */
    /* ... */
}

int verify_password() { /* returns 1 if OK, 0 if wrong */
    /* ... */
    return 1;
}
```

5.2 char arrays = strings

In C there is no separate string type. Text lives in **char name[50]** — up to 49 letters plus end marker. Use **strcpy** to copy text and **strcmp** to compare.

C

```
strcpy(product_name[0], "Rice");
if (strcmp(password, "1122") == 0) { /* 0 means equal */
    return 1;
}
```

5.3 2D arrays — stock grid

dealer_stock[dealer][product] is a table: row = which dealer, column = which product. Example: **dealer_stock[0][0]** = Dhaka Dealer's Rice count.

Index	Meaning	Start value
dealer_stock[0][0]	Dhaka Dealer — Rice	20
dealer_stock[0][1]	Dhaka Dealer — Oil	15
dealer_stock[1][0]	Chittagong Dealer — Rice	25

2D array examples from `init_data()`

5.4 extern — one copy of data for all files

Arrays are **defined once** in data.c. data.h says **extern** so company.c, dealer.c, and retailer.c all use the **same** stock numbers. When a dealer orders, company stock changes everywhere immediately.

C

```
/* data.h */
extern int company_stock[MAX_PRODUCTS];

/* data.c */
int company_stock[MAX_PRODUCTS]; /* the real memory */
```

5.5 break and return

- **break** — stop a for loop early when you found the right ID.
- **return** — leave a function immediately (used after errors so bad orders don't run).

6. The Data Layer (data.c)

6.1 Sample data at startup

`init_data()` fills the system before any menu appears. No file reading — everything is hard-coded for learning.

Products	Company stock
1 Rice	100
2 Oil	80
3 Sugar	60

Products

Dealer ID	Name
1	Dhaka Dealer
2	Chittagong Dealer

Dealers

Retailer ID	Name	Linked dealer
1	Mirpur Shop	Dealer 1
2	Gulshan Shop	Dealer 2

Retailers

Product	Company	Dhaka	Chittagong	Mirpur	Gulshan
Rice	100	20	25	5	4
Oil	80	15	12	3	6
Sugar	60	10	18	2	1

Full stock matrix at program start

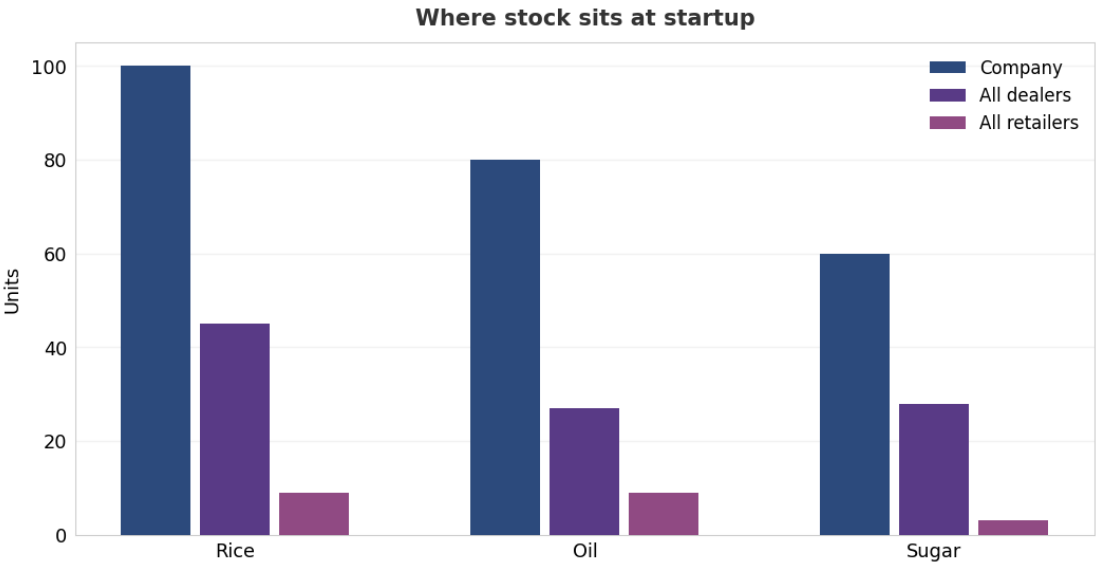


Figure 2: Most units live at the company — like a real warehouse

7. main.c — Program Boss

Every C program starts at **main()**. DMS loads data, then loops on the main menu until you choose Exit (0).

C

```
int main() {
    int choice;
    init_data();

    do {
        print_menu_header("DISTRIBUTION MANAGEMENT SYSTEM");
        print_menu_item(1, "Company Login");
        print_menu_item(2, "Dealer Login");
        print_menu_item(3, "Retailer Login");
        print_menu_item(0, "Exit");
        printf("\nEnter your choice: ");
        scanf("%d", &choice);

        if (choice == 1)      company_menu();
        else if (choice == 2) dealer_menu();
        else if (choice == 3) retailer_menu();
        else if (choice != 0)
            print_error("Invalid choice. Please try again.");
    } while (choice != 0);

    return 0;
}
```

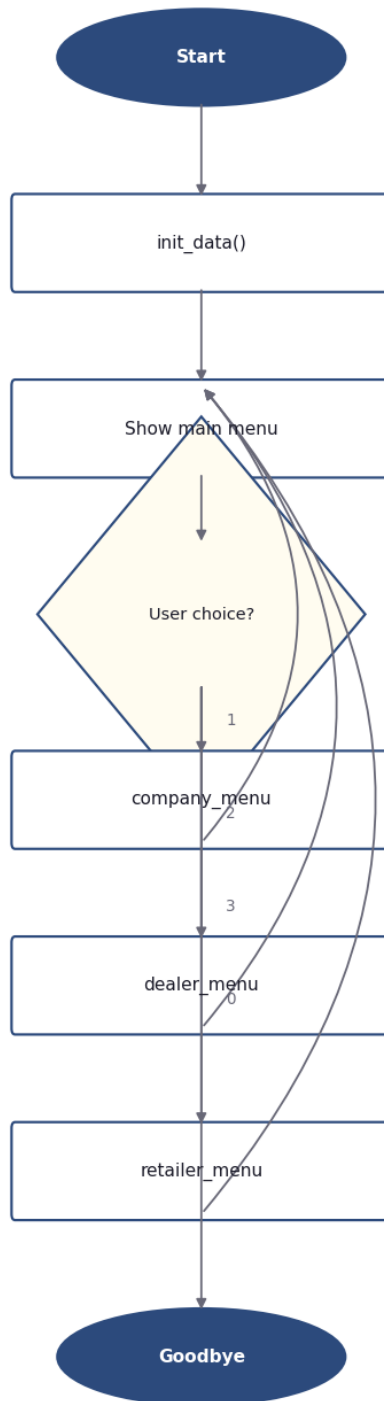


Figure 3: Main program flow

8. display.c — Screens and Tables

All pretty printing lives here. Other files call these helpers so menus look the same everywhere.

- **print_menu_header / print_menu_item** — boxed menus.
- **print_table_product_row** — one line of a stock table.
- **verify_password()** — compares input to 1122.
- **print_error / print_success** — user feedback.
- **print_order_summary** — after a successful order.

C

```
void print_table_product_row(int id, char name[], int stock) {  
    printf("| %-4d | %-20s | %-10d |\n", id, name, stock);  
}  
/* %-20s pads name to 20 chars so columns align */
```

9. company.c — Company Panel

Company logs in with password only. It **views** data — no ordering in this version.

Menu	Action
1	List all dealers
2	List all products
3	Show company warehouse stock
0	Back to main menu

Company menu

list_dealers() uses a for loop — same pattern you already know:

C

```
for (i = 0; i < dealer_total; i++) {  
    print_table_dealer_row(dealer_id[i], dealer_name[i]);  
}
```


10. dealer.c — Dealer Panel

10.1 Login

1. Show dealer table.
2. User enters Dealer ID.
3. Loop searches dealer_id[] to find matching index.
4. Password 1122, then dealer menu opens.

10.2 The found pattern (used everywhere)

```
c
found = 0;
for (i = 0; i < product_total; i++) {
    if (product_id[i] == product_id_input) {
        found = 1;
        product_index = i;
        break;
    }
}
if (found == 0) {
    print_error("Product not found.");
    return;
}
```

10.3 Ordering from company

Three checks before stock moves: product exists, quantity > 0, company has enough stock.
Then two lines of math:

```
c
company_stock[product_index] -= quantity;
dealer_stock[dealer_index][product_index] += quantity;
```

Example: Company Rice 100 → 90, Dhaka Dealer Rice 20 → 30 after ordering 10 units.

11. retailer.c — Shop Panel

Same ideas as dealer, but retailers order from their **linked dealer** (retailer_dealer[]). Stock moves from dealer_stock down to retailer_stock.

Retailer	Buys from
Mirpur Shop (ID 1)	Dhaka Dealer
Gulshan Shop (ID 2)	Chittagong Dealer

Retailer–dealer links

c

```
dealer_stock[dealer_index][product_index] -= quantity;
retailer_stock[retailer_index][product_index] += quantity;
```

12. Full Story With Numbers

Follow one product (Rice) through two orders.

Step A — Dealer orders 10 Rice from company

Location	Before	After
Company Rice	100	90
Dhaka Dealer Rice	20	30

After dealer order

Step B — Mirpur Shop orders 5 Rice from dealer

Location	Before	After
Dhaka Dealer Rice	30	25
Mirpur Shop Rice	5	10

After retailer order

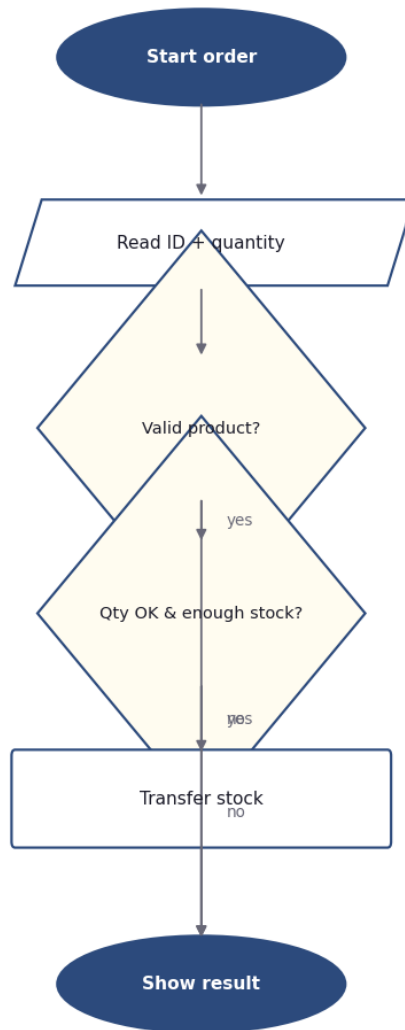


Figure 4: Every order follows this validation chain

13. Common Mistakes

Problem	Fix
Compile error / missing symbols	Include ALL six .c files in gcc command
Wrong password	Use 1122
Dealer not found	Use ID 1 or 2 from the table
Not enough stock	Order a smaller quantity
Product not found	Use product ID 1, 2, or 3

Troubleshooting

14. Practice Exercises

Easy

- Change the password in `display.c` and test login.
- Add a 4th product in `init_data()` (e.g. Salt).

Medium

- Add menu option: dealer sees total units across all products.
- Print which dealer a retailer is linked to on login.

Hard

- Save stock to a file on exit and load on start (needs file I/O).
- Reject orders that would leave seller with zero stock of everything.

15. Glossary

Word	Simple meaning
DMS	Distribution Management System — this project
Compile	Turn .c source into runnable program
Executable	The dms file you run with ./dms
Header (.h)	List of names other files can use
extern	This variable lives in another .c file
2D array	Grid: rows and columns like a spreadsheet
Stock	How many units someone holds
Index	Array position (starts at 0)
ID	User-facing number (often 1, 2, 3...)

Quick reference

Cheat sheet

BASH

```
gcc main.c data.c display.c company.c dealer.c retailer.c -o dms
./dms
Password: 1122
Dealer IDs: 1, 2 | Retailer IDs: 1, 2 | Product IDs: 1, 2, 3
```

You now know how DMS works from compile command to stock transfer. Open each .c file and match what you read here to the real code line by line.